Additional Tools and Routing Exercises

Aims

Develop a deeper understanding of routing mechanisms in Symfony

Objectives

- Set up and learn about more development and debugging tools
- Create different types of routes to match application specific patterns

Additional Tools

Symfony Profiler
composer require profiler --dev

adds the Symfony Profiler bar at the bottom of our pages

only added for development (i.e. not not used on production)

to debug your controllers and show variables etc:

dump() (equivalent of var_dump() for Symfony apps)

dd() (dump and die)

in Twig templates (consider that e.g. an array was handed over to the template):

{% dump array %} -> displayed in web debug toolbar

{{ dump(foo.bar) }} -> dumps inline (HTML output)

Flex
Symfony Flex is a Composer plugin that modifies the behaviour of require, update, and remove.

- Aliases

- Recipes

Above alias "profiler" is resolved to download a number of bundles.

Bundles can then be enabled and configures by means of recipes.

Debug Package
composer require debug

adds a logging facility (see var/log/dev.log)

allows dumped information to be viewed in the Profiler as well as on the console (via php bin/console server:dump)

XDebug
Requires three components to work together:

PHP with XDebug extension

- https://xdebug.org/wizard (https://xdebug.org/wizard)
- in php.ini
    - zend_extension = php_xdebug-xyz123.dll
    - xdebug.remote_enable = 1
    - xdebug.remote_host = "localhost"
    - xdebug.remote_port = 9000
    - xdebug.mode=debug
- check via php -v (should say with XDebug)

Browser plugin e.g. Chrome with Xdebug helper

IDE such as PhpStorm with XDebug enabled/configured

Allows debugging of client-server web applications just like desktop software (breakpoints, step-into, call stack, object inspector etc.)

Exercises

Routing (Symfony Docs):

https://symfony.com/doc/current/routing.html (https://symfony.com/doc/current/routing.html)

1. Static patterns
Create a new route in your app to match the following static pattern:

http://localhost:8000/hello/world

This route should be defined using annotations (PHP 7.x and below) or PHP attributes (PHP 8.x and above). Create a new action helloAction in your default Controller to be called by this route. Make the new action render a hello.html.twig template to display the message "Welcome to AWD".

2. Wildcard patterns
Create a new route but this time in config/routes.yaml, matching the following pattern:

http://localhost:8000/hello/name

where name can be anything. As before, create a new action and a corresponding template, this time to display Hello followed by the name from the URL.

3. Regular expressions
Create a new route matching the pattern:

http://localhost:8000/increment/1

allowing any single digit (0..9) in the last part. Have a look here for help with regular expressions: https://regex101.com/ (https://regex101.com/)

Create a new action and a corresponding template, this time to display the value of the digit + 1.

4. More regular expressions
Create a new route similar to the one in 2. but this time allow one of the three salutations hello/hi/welcome in the middle part and a string in the last part:

E.g.

http://localhost:8000/hello/Sam

http://localhost:8000/hi/John

The response should contain both, the salutation and string (e.g. hello Sam).

5. More Twig - Working with two-dimensional arrays

In one of your controller actions, create a two-dimensional array contacts to store name and email address of a number of people:

Jane Smith	jane@smith.com
John Smith	john@smith.com
Bill Gates	bill@microsoft.com

Hand the array over to a Twig template and produce a table with all names and email addresses in the output.

6. More Debugging Tools - Symfony Profiler

To test debugging, dump the array contacts from above to be displayed by the Symfony Profiler. Make sure you have the required dependencies installed and that you are producing valid HTML in order for the Symfony Profiler to be included at the bottom of your page.

7. Even more Debugging Tools - XDebug

Install XDebug (PHP extension) and Xdebug helper (browser extension). Configure PhpStorm to work with XDebug and to accept external connections.

Add a break point in PhpStorm at the line that renders your template and with debugging enabled refresh your browser. What is the name of the PHP class that contains the render() function?